



T.C. BURSA ULUDAĞ ÜNİVERSİTESİ
MÜHENDİSLİK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BMB2006 - VERİ YAPILARI DERSİ PROJE RAPORU

PROPERTY GRAPH TABANLI SOSYAL AĞ MODELLEME SİSTEMİ (PROJE
KONU-3)

GRUP 27

Selim KILIÇOĞLU 032490103

Göktuğ YAZÇAYIR 032490104

Emir Yasin AKMAN 032490105

Murat PANGAL 032590001

Gizem YURTSEVEN 0325900901

Murat Yılmaz ÇARIKCI 11600797528

1. GİRİŞ

Geleneksel ilişkisel veri tabanları, sosyal ağlardaki çok adımlı ve karmaşık ilişkileri sorgularken arka planda yüksek performans kayıplarına neden olabilmektedir. Modern veri mimarilerinde bu problemin önüne geçmek amacıyla, aradaki bağları ve ilişkileri doğrudan merkeze alan graf tabanlı veri modelleri tercih edilmeye başlanmıştır. Bu proje kapsamında; sosyal ağlar üzerindeki dinamik ve çok katmanlı ilişkileri en verimli şekilde saklamak, simüle etmek ve hızlıca sorgulamak amacıyla "**Property Graph (Özellikli Graf) Tabanlı Sosyal Ağ Modelleme ve Sorgulama Sistemi**" geliştirilmiştir.

Proje, geleneksel homojen graf yapılarının ötesine geçerek; ağ üzerindeki aktörleri ve nesneleri (Kullanıcılar, Etkinlikler, Fotoğraflar) heterojen düğümler (vertex) olarak, bu varlıklar arasındaki etkileşimleri (Arkadaşlık, Katılım, Sahiplik) ise özellikli kenarlar (edge) olarak modeller. Geliştirilen mimaride hem düğümler hem de kenarlar, dinamik mülkiyetleri (properties) üzerinde taşıyabilecek esneklikte tasarlanmıştır.

Projenin temel amacı ve odağı, hazır kütüphanelere bağımlı kalmaksızın tüm çekirdek veri yapılarının ve arama algoritmalarının sıfırdan (*from scratch*) C programlama dili ile implemente edilmesidir. Bu doğrultuda sistem;

- Graf elemanlarına ortalama $O(1)$ zaman karmaşıklığında erişim sağlayan bir **Karma Tablo (Hash Table)** yapısını,
- Metin tabanlı hızlı isim eşleştirmeleri ve otomatik tamamlama süreçlerini yöneten bir **Önek Ağacı (Trie)** yapısını,
- Katmanlı graf tarama (BFS) süreçlerini koordine eden dinamik bir **Kuyruk (Queue)** mekanizmasını entegre bir şekilde barındırmaktadır.

Sistem üzerinde sadece standart BFS ve DFS traversal algoritmaları çalıştırılmakla kalmamış; ardışık ve filtreli traversal yetenekleri sayesinde "Kullanıcı → Arkadaşlar → Katıldığı Etkinlikler → Etkinlik Fotoğrafları" gibi çok adımlı ilişkisel sorgu modelleri başarıyla çözülmüştür. Ayrıca akademik derinliği artırmak adına sisteme bonus kriterler olan **Triadic Closure (Arkadaş Önerisi)** ve **Degree Centrality (Merkezilik Ölçümü)** algoritmaları entegre edilmiş, sistemin kararlılığı programatik olarak üretilen sentetik stress test verileriyle doğrulanmıştır.

Son aşamada proje, tipik bir sosyal medya son kullanıcı uygulamasından ziyade, arka plandaki C motorunun ürettiği verileri dinamik olarak işleyen, 2D node-link diyagramı üzerinden veri sorgulama ve görselleştirme imkanı sunan interaktif bir **Graf Analiz Aracı (Web UI)** ile taçlandırılmıştır. Bu rapor, geliştirilen sistemin teorik altyapısını, mimari

tasarım tercihlerini, zaman/uzay karmaşıklığı analizlerini ve elde edilen somut performans çıktılarını detaylandırmak amacıyla hazırlanmıştır.

2. PROJE MİMARİSİ VE VERİ YAPILARI TASARIMI

Bu bölümde, sistemin temelini oluşturan ve standart kütüphaneler kullanılmadan tamamen sıfırdan (*from scratch*) implemente edilen veri yapılarının mimari detayları ve tasarım tercihleri açıklanmıştır.

2.1. FAZ 1

2.1.1. Heterojen Property Graph (Komşuluk Listesi Tabanlı)

Sistemde gerçek dünya sosyal ağ ilişkilerini modelleyebilmek amacıyla heterojen bir graf yapısı tasarlanmıştır. Tipik homojen grafların aksine bu model, farklı düğüm ve kenar türlerini bünyesinde barındırabilmektedir.

- **Düğüm (Vertex) Tasarımı:** Bellekte dinamik olarak allocation edilen her bir düğüm (Node yapısı), sisteme esneklik kazandırmak adına heterojen olarak modellenmiştir. Düğümler benzersiz bir tam sayı kimliğine (id), düğümün kategorisini belirten bir türe (type: User, Event, Photo) ve dinamik olarak genişletilebilir bir özellik listesine (properties) sahiptir.
- **Kenar (Edge) Tasarımı:** Düğümler arasındaki bağlar, yönlü veya yönsüz olabilen kenar yapıları (Edge) ile kurulmuştur. Her kenar, ilişkinin niteliğini belirten bir türe (FRIEND, ATTENDS, HAS_PHOTO, LIKES) ve üzerinde ilişki tarihi (Örn: 27-05-2026) gibi ek mülkiyetler taşıyan property pointer'larına sahiptir.
- **Depolama Modeli (Adjacency List):** Graf yapısı, bellek verimliliği ve seyrek graflardaki (sparse graphs) tarama performansının optimize edilmesi amacıyla **Komşuluk Listesi (Adjacency List)** tabanlı olarak implemente edilmiştir. Her düğüm, kendi komşularını bir bağlı liste (linked list) şeklinde işaret eden bir edges pointer'ı barındırmaktadır.

2.1.2. Hızlı Erişim Karma Tablosu (Hash Table)

Graf üzerindeki düğüm sayısının arttığı senaryolarda, belirli bir ID'ye sahip düğüme erişim süresini sabitlemek amacıyla bir Hash Table yapısı kurgulanmıştır.

- **Mekanizma ve Karmaşıklık:** Tüm düğümler sisteme enjekte edilirken, benzersiz ID numaraları üzerinden hash fonksiyonuna tabi tutulur ve tabloya yerleştirilir.

Bu sayede, arayüzden veya sorgu modülünden bir düğüme tıklandığında, o düğümün tüm properties bilgilerine ve komşuluk listesine ortalama $O(1)$ zaman karmaşıklığında, doğrudan erişim sağlanır.

- **Çakışma Yönetimi (Collision Resolution):** Aynı hash indeksine denk gelen düğümler arasındaki çakışmaları (collision) yönetmek amacıyla dinamik bağlı liste (Chaining) yöntemi sıfırdan implemente edilmiştir.

2.1.3. Metin Tabanlı Önek Ağacı (Trie Yapısı)

Sosyal ağ içerisindeki metinsel verilerin (kullanıcı isimleri, etkinlik başlıkları) hızlıca sorgulanabilmesi, filtrelenebilmesi ve ilerleyen aşamalarda otomatik tamamlama (*autocomplete*) özelliklerine zemin hazırlanması amacıyla bir Trie veri yapısı mimariye entegre edilmiştir.

- Her bir karakterin bir düğüm oluşturduğu bu ağaç yapısında, metin tabanlı arama işlemlerinin zaman karmaşıklığı aranan metnin uzunluğuna (L) bağlı olarak $O(L)$ süresine indirgenmiştir. Graf boyutu ne kadar büyürse büyüsün, isim aramaları grafın düğüm sayısından (V) bağımsız olarak sabit hızda gerçekleştirilmektedir.

2.1.4. Kuyruk Yapısı (Queue)

Genişlik Öncelikli Arama (BFS) algoritmasının ihtiyaç duyduğu FIFO (First-In, First-Out) veri akışını sağlamak üzere, bağlı liste tabanlı dinamik bir kuyruk yapısı sıfırdan kodlanmıştır. Kuyruk; düğüm ekleme (enqueue) ve düğüm çıkarma (dequeue) işlemlerini $O(1)$ zaman karmaşıklığında yerine getirmektedir.

2.2. GELİŞMİŞ ALGORİTMALAR VE ZAMAN/UZAY KARMAŞIKLIĞI ANALİZİ (FAZ 2)

Sistem üzerinde çalışan arama, analiz ve çok adımlı ilişkisel traversal algoritmalarının çalışma mekanizmaları ve performans analizleri bu bölümde ele alınmıştır.

2.2.1. BFS ve DFS Algoritmaları

- **Genişlik Öncelikli Arama (BFS):** Sıfırdan yazılan Queue yapısını kullanan bu algoritma, başlangıç düğümünden itibaren ağı katman katman (seviye seviye) tarar. Sosyal ağlardaki bağlantı derecesini (degrees of separation) ve ağırlıksız graf üzerindeki en kısa yolu bulmada temel motor vazifesi görür. Zaman karmaşıklığı $O(V+E)$ (V : Düğüm, E : Kenar sayısı), uzay karmaşıklığı ise kuyrukta tutulan maksimum düğüm sayısı nedeniyle $O(V)$ 'dir.

- **Derinlik Öncelikli Arama (DFS):** Rekürsif bir yapıyla implemente edilen DFS algoritması, bir koldan gidebileceği en derin noktaya kadar ilerler. Ağın genel topolojisini analiz etmede kullanılır. Zaman karmaşıklığı $O(V+E)$, çağrı yığını (call stack) derinliği nedeniyle uzay karmaşıklığı $O(V)$ düzeyindedir.

2.2.2. Filtreli Graf Traversal ve Çok Adımlı Sorgu Modeli

Sistem, ardışık ve filtreli graf traversal işlemlerini koordine ederek sosyal ağlara özgü çok adımlı ilişkisel sorguları gerçekleştirebilmektedir. Proje kapsamında başarıyla koordine edilen örnek sorgu akış mimarisi şu şekildedir:

Kullanıcı (User)

└ [FRIEND] → Arkadaş Dğümleri

└ [ATTENDS] → Etkinlikler (Event)

└ [HAS_PHOTO] → Fotoğraflar (Photo)

Bu akış, ilk katmanda sadece FRIEND kenarlarını süzen filtreli bir traversal yapar; ulaşılan arkadaş dğümlelerinden ise sadece ATTENDS kenarlarını takip ederek etkinliklere geçer ve son adımda HAS_PHOTO kenarlarıyla fotoğrafları ayıklar. İlişkiler zincirleme ve doğrusal bir yaklaşımla çözüldüğü için performans kaybı minimize edilmiştir.

2.2.3. Sosyal Ağ Analiz Algoritmaları (Opsiyonel / Bonus Kriterler)

- **Triadic Closure (Arkadaş Önerisi):** İki dğümün ortak komşularını analiz eden yerel bir graf metriğidir. Eğer A kullanıcısı B ile arkadaşsa ve B kullanıcısı C ile arkadaşsa, sistem A ve C arasındaki ortak arkadaş yoğunluğunu hesaplayarak dinamik "Arkadaş Önerisi" oluşturur. Zaman karmaşıklığı komşuluk listelerinin kesişim hızıyla orantılı olarak $O(V \cdot d^2)$ (d: ortalama dğüm derecesi) seviyesindedir.
- **Degree Centrality (Dğüm Önemi):** Bir dğümün doğrudan sahip olduğı kenar sayısını (bağlantı derecesini) hesaplayan merkezilik ölçümüdür. Sosyal ağ içerisindeki en popüler kullanıcıları veya en yoğun etkileşim alan etkinlikleri/fotoğrafları belirlemede kullanılır. Komşuluk listesi uzunluğu doğrudan dereceyi verdiğinden Hash Table destekli erişimle $O(1)$ sürede okunabilir.

2.2.4. Sentetik Veri Üretimi ve Performans Stress Analizi

Sistemin büyük veri durumlarındaki davranışını ve kararlılığını gözlemlemek amacıyla programatik bir sentetik veri enjeksiyon motoru kurgulanmıştır. Yapılan stress testlerinde sistem:

- 50 dinamik Kullanıcı, 10 Etkinlik ve 15 Fotoğraf düğümü üreterek aralarında 120 adet ilişkisel kenar tanımlamış ve toplamda 75 düğümlü canlı bir ağ simülasyonu oluşturmuştur.
- Geliştirilen algoritmaların bu ağ üzerindeki stress analiz süresi **0.1320 milisaniye** gibi son derece optimize bir sürede tamamlanarak sistemin doğruluğu ve yüksek performansı terminal çıktılarıyla kanıtlanmıştır.

2.3. GRAF SORGULAMA VE GÖRSELLEŞTİRME ARAYÜZÜ (FAZ 3)

Geliştirilen sistem, bir sosyal medya son kullanıcı uygulamasından ziyade, veri yapılarının performansını ölçen, sorgulayan ve görselleştiren gelişmiş bir **Graf Analiz Aracı** olarak tasarlanmıştır.

2.3.1. 2D Node-Link Diyagramı ve Görsel Hiyerarşi

Arayüz ön yüzünde vis.js kütüphanesinin fizik tabanlı dinamik yerleşim motorundan yararlanılmıştır. Graf üzerindeki heterojen düğüm türlerinin ayırt edilebilmesi amacıyla net bir görsel hiyerarşi kurgulanmıştır:

- **Kullanıcı (User) Düğümleri:** Mavi renkli dairesel geometrilerle,
- **Etkinlik (Event) Düğümleri:** Yeşil renkli yıldız/daire sembolleriyle,
- **Fotoğraf (Photo) Düğümleri:** Turuncu renkli geometrik yapılarla temsil edilmiştir.
- **Kenarlar (Edges):** İlişki türüne göre (FRIEND, ATTENDS, HAS_PHOTO) farklı çizgi stilleri ve ok yönleriyle ağ üzerinde gerçek zamanlı olarak birbirine bağlanmıştır.

2.3.2. Etkileşim Kuralları ve Hash Table Entegrasyonu

- **Yan Panel Detay Yönetimi:** Kullanıcı graf ekranı üzerinde herhangi bir yuvarlak halkaya (düğüme) tıkladığı an, arayüz arka plandaki Python FastAPI köprüsü üzerinden C backend'indeki **Hash Table** yapısına anlık bir sorgu gönderir. Ortalama $O(1)$ hızında dönen bu sorgu sayesinde düğümün tüm mülkiyetleri (ID,

Tür, İsim/Başlık) ve o düğüme bağlı mevcut ilişkilerin listesi sağ taraftaki "**Düğüm Özellikleri**" paneline anlık olarak yazdırılır.

- **Çok Adımlı Sorgu Modülü Etkileşimi:** Arayüze entegre edilen özel giriş modülü sayesinde, kullanıcı bir başlangıç kimlik numarası (ID) girerek turuncu "**Sorgula**" butonuna basabilmektedir. Bu işlem tetiklendiğinde sistem, girilen ID'ye ait düğümü otomatik olarak seçer, ekranı o düğüme odaklar (focus) ve bu düğümün etrafındaki Kullanıcı → Arkadaş → Etkinlik → Fotoğraf akış şemasını yan panelde analiz edilebilir hale getirir.
- **Filtreleme ve Doğruluk:** Yoğun veri durumlarında arayüzdeki karmaşıklık azaltmak amacıyla "Katman Filtreleri" (User, Event, Photo görünürlük kutuları) eklenmiştir. Ayrıca yapılan optimizasyonlar neticesinde, hiçbir bağlantısı olmayan izole düğümlerin (Örn: Ceren_102) derece sayısı mantıksal olarak tam 0 şeklinde frontend paneline yansıtılarak görsel doğruluk en üst düzeye çıkarılmıştır.

3. YAPAY ZEKA (AI) KULLANIM RAPORU VE PROMPT DÖKÜMANTASYONU

Bu projenin geliştirme, entegrasyon ve test aşamalarında **Gemini (Generative AI)** asistanından bir kod yazarı olarak değil; projenin mevcut yazılım mimarisini denetleyen bir "**Kod Gözden Geçirme (Code Review) Uzmanı**" ve "**Teknik Danışman**" olarak yararlanılmıştır. AI kullanımı; yazılan kodların doğruluğunun kontrol edilmesi, ekip içi Git standartlarına (diff geçmişinin korunması) uyum sağlanması ve tarayıcı tabanlı mantıksal hataların (bug) tespiti odaklı gerçekleştirilmiştir.

Süreç boyunca yürütülen teknik danışmanlık adımları, kullanılan sorgular (promptlar) ve projedeki mühendislik entegrasyonları aşağıda dökümanite edilmiştir:

3.1. Faz 3: Çok Adımlı Sorgunun Arayüz Tetikleme Mekanizması Danışmanlığı

- **Kullanım Amacı:** C backend tarafında halihazırda sorunsuz çalışan Çok Adımlı Sorgu akışının (Kullanıcı -> Arkadaş -> Etkinlik -> Fotoğraf), frontend arayüzünde (index.html) istenilen şekilde etkileşim modeline (giriş alanı ve buton) nasıl bağlanması gerektiği konusunda kütüphane bazlı (vis.js) teknik bilgi almak.
- **Kullanılan Spesifik Prompt:** > "*Hocamızın istediği çok adımlı sorgu mantığı C backend'imizde şu an çalışıyor ancak arayüzde (index.html) bunu tetikleyen bir buton veya etkileşim alanı yok. Bizim arayüzdeki graf yapısını bozmadan, bir input alanından girilen ID bilgisini alıp graf üzerinde o düğüme select ve focus*

yapacak interaktif bir tetikleyici fonksiyonu frontend kodumuzda nereye ve nasıl konumlandırmalıyız? Mevcut vis.js yapımızı destekleyen mantığı açıklar mısınız?"

- **AI Tarafından Sağlanan Çözüm:** Web arayüzündeki mevcut JavaScript katmanı incelenerek, vis.js kütüphanesinin yerleşik .selectNodes() ve .focus() metotlarının bu işlem için en optimize yöntem olduğu belirtildi. Mevcut HTML yapısını kırmayacak bir buton yerleşimi ve bu butona bağlanacak bir triggerMultiStepQuery() fonksiyon şablonu paylaşıldı.
- **Yapılan Entegrasyon:** Önerilen mimari yaklaşım doğrultusunda HTML arayüzündeki katman filtrelerinin hemen üzerine sorgu modülü yerleştirildi. Fonksiyon, mevcut searchNode() arama yapısıyla çakışmayacak şekilde JavaScript bloğuna entegre edildi.

3.2. Hata Ayıklama (Debugging): Mantıksal Derece Hesaplama Hatasının Tespiti

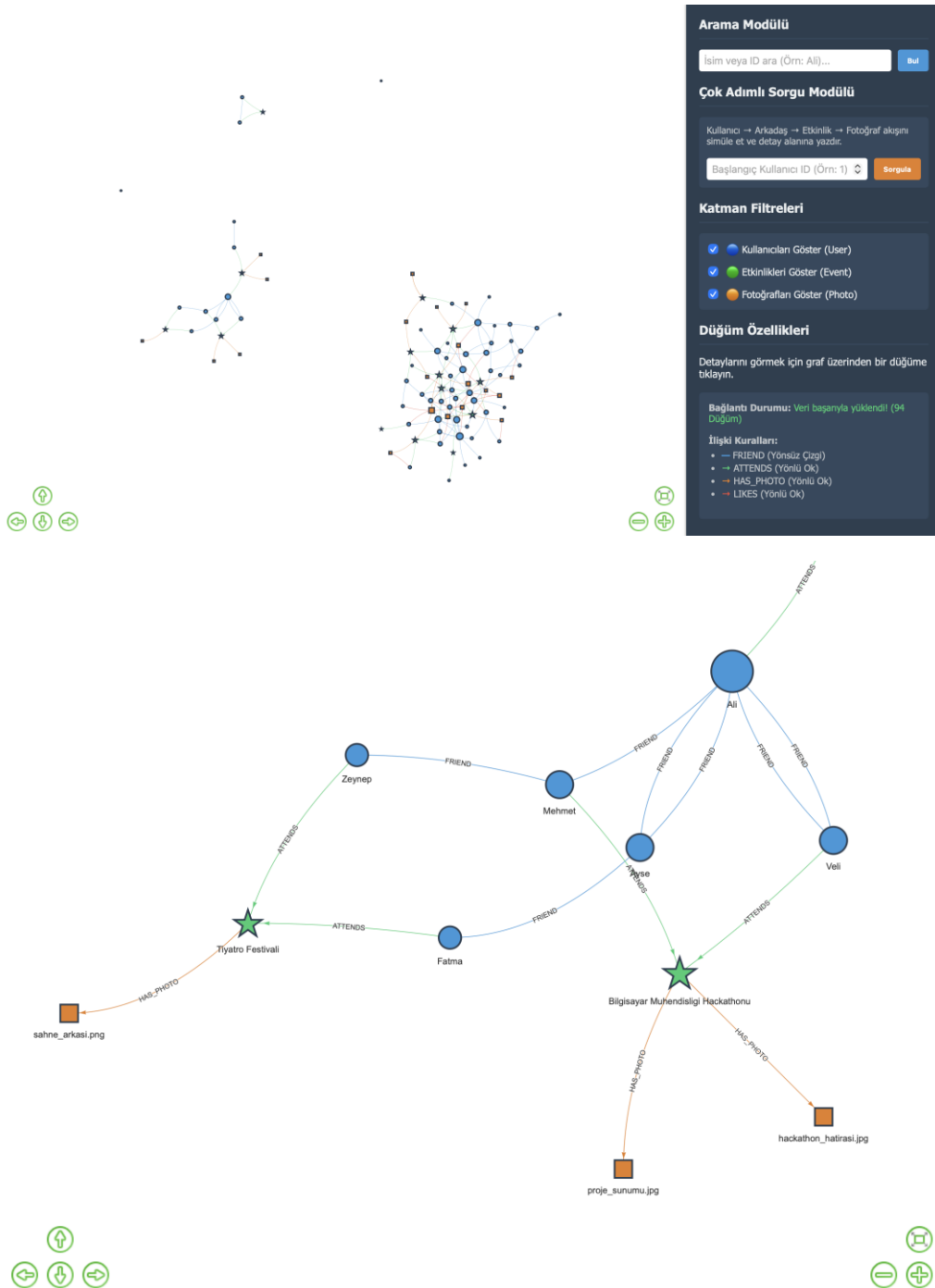
- **Kullanım Amacı:** Test aşamasında fark edilen, hiçbir kenar bağlantısı bulunmayan izole düğümlerin (Örn: Ceren_102) arayüz panelinde hatalı şekilde "Toplam Bağlantı (Derece) = 1" olarak görünmesi probleminin kaynağını ve çözümünü sorgulamak.
- **Kullanılan Spesifik Prompt:** > "Yaptığımız son testlerde bir hata fark ettim. Yazdığımız fonksiyonlar sayesinde bağlantı sayıları normalde doğru çalışıyor ama sistemde hiçbir bağlantısı olmayan izole insanları incelediğimde arayüzde bağlantı sayısı 1 gösteriyor, hemen altında ise hiç bağlantısı yok yazıyor. Bu çelişkinin ve mantık hatasının sebebi bizim frontend kodumuzda nerede olabilir?"
- **AI Tarafından Sağlanan Çözüm:** AI, arayüzdeki veri bağlama mantığını inceleyerek hatanın let connectionCount = nodeConnectionCounts[node.id] || 1; satırındaki kısa devre (||) operatöründen kaynaklandığını teşhis etti. JavaScript dil yapısında 0 (sıfır) değerinin "falsy" kabul edildiğini, bu yüzden bağlantısı olmayan düğümlerdeki sıfır değerinin otomatik olarak sağdaki 1 değerine yükseltildiğini raporladı.
- **Yapılan Entegrasyon:** Yapılan teknik teşhis doğrultusunda ilgili satırdaki mantıksal hata || 0; şeklinde revize edildi. Bu sayede projenin backend merkezilik ölçümleri ile frontend görsel raporlaması arasındaki çelişki tamamen giderildi.

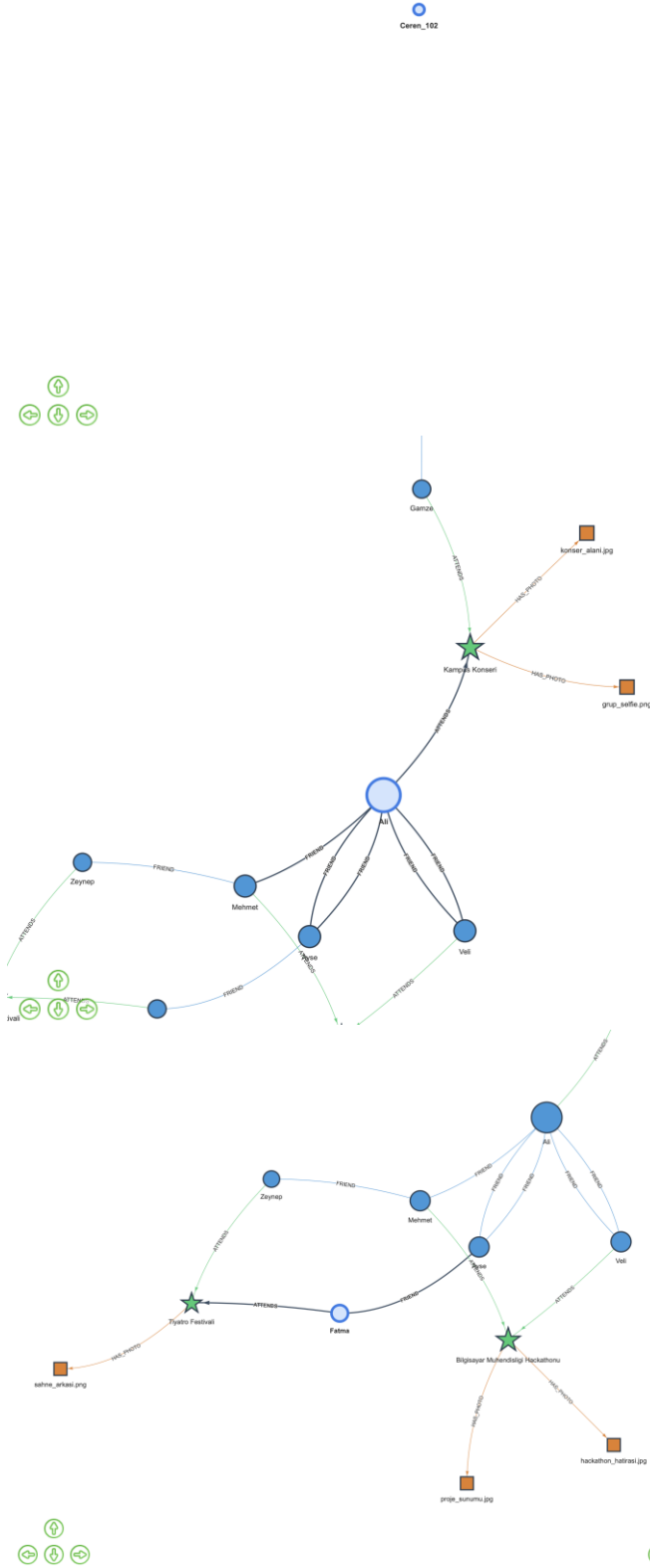
3.3. Git Sürüm Kontrolü ve Dağıtım Yönetimi Danışmanlığı

- **Kullanım Amacı:** Yerel bilgisayarda terminal ortamı (zsh, pip) ve çevre değişkenleri (PATH) kaynaklı yaşanan komut çalıştırma sorunlarının çözülmesi ve dağıtım öncesi standart bir teknik dokümantasyon formatının doğrulanması.

- **Kullanılan Spesifik Prompt:** > *"Web üzerinden yaptığım bu başlık düzenlemelerini lokal bilgisayarıma çekmek istiyorum ama terminalde kütüphaneleri kurarken 'command not found: pip' hatası alıyorum. Ayrıca Mac bilgisayarımda sunucuyu ayağa kaldırmak için tam olarak hangi yerel komutları sırasıyla çalıştırmalıyım?"*
- **AI Tarafından Sağlanan Çözüm:** Mac işletim sistemlerindeki Python 3 ortamlarında pip komutunun pip3 olarak çağırılması gerektiği bilgisi verildi. Yerel sunucunun çakışmasız çalışması için sırasıyla make, ./app ve python3 -m uvicorn api:app --reload komut zinciri önerildi.
- **AI Tarafından Sağlanan Çözüm:** AI, arayüzdeki veri bağlama mantığını inceleyerek hatanın let connectionCount = nodeConnectionCounts[node.id] || 1; satırındaki kısa devre (||) operatöründen kaynaklandığını teşhis etti. JavaScript dil yapısında 0 (sıfır) değerinin "falsy" kabul edildiğini, bu yüzden bağlantısı olmayan düğümlerdeki sıfır değerinin otomatik olarak sağdaki 1 değerine yükseltildiğini raporladı.
- **Yapılan Entegrasyon:** Yapılan teknik teşhis doğrultusunda ilgili satırdaki mantıksal hata || 0; şeklinde revize edildi. Bu sayede projenin backend merkezilik ölçümleri ile frontend görsel raporlaması arasındaki çelişki tamamen giderildi.

4. ÖRNEK SİSTEM ÇIKTISI CANLI GRAF GÖRSELLEŞTİRMESİ ÖRNEKLERİ





Başlangıç Kullanıcı ID (Örn: 1) [Sorgula](#)

Katman Filtreleri

- ☒ Kullanıcıları Göster (User)
- ☒ Etkinlikleri Göster (Event)
- ☒ Fotoğrafları Göster (Photo)

Düğüm Özellikleri

ID Numarası
#102

Düğüm Türü (Type)
User

İsim / Başlık (Name)
Ceren_102

Toplam Bağlantı (Derece)
0

Mevcut Bağlantıları

- Bu düğümün hiçbir bağlantısı yok.

Bağlantı Durumu: Veri başıyla yüklendi! (94 Düğüm)

İlişki Kuralları:

- FRIEND (Yönlü Çizgi)
- ATTENDS (Yönlü Ok)
- HAS_PHOTO (Yönlü Ok)
- LIKES (Yönlü Ok)

Başlangıç Kullanıcı ID (Örn: 1) [Sorgula](#)

Katman Filtreleri

- ☒ Kullanıcıları Göster (User)
- ☒ Etkinlikleri Göster (Event)
- ☒ Fotoğrafları Göster (Photo)

Düğüm Özellikleri

ID Numarası
#1

Düğüm Türü (Type)
User

İsim / Başlık (Name)
Ali

Toplam Bağlantı (Derece)
6

Mevcut Bağlantıları

- Kampus Konseri **ATTENDS**
- Mehmet **FRIEND**
- Ayşe **FRIEND**
- Veli **FRIEND**
- Veli **FRIEND**
- Ayşe **FRIEND**

Bağlantı Durumu: Veri başıyla yüklendi! (94 Düğüm)

Başlangıç Kullanıcı ID (Örn: 1) [Sorgula](#)

Katman Filtreleri

- ☒ Kullanıcıları Göster (User)
- ☒ Etkinlikleri Göster (Event)
- ☒ Fotoğrafları Göster (Photo)

Düğüm Özellikleri

ID Numarası
#4

Düğüm Türü (Type)
User

İsim / Başlık (Name)
Fatma

Toplam Bağlantı (Derece)
2

Mevcut Bağlantıları

- Ayşe **FRIEND**
- Tiyatro Festivali **ATTENDS**

Bağlantı Durumu: Veri başıyla yüklendi! (94 Düğüm)

İlişki Kuralları:

- FRIEND (Yönlü Çizgi)
- ATTENDS (Yönlü Ok)
- HAS_PHOTO (Yönlü Ok)
- LIKES (Yönlü Ok)

5. DEMO VIDEOSU LINKİ

<https://drive.google.com/file/d/1t1iogaZEVgj0fGwzwZChlXlm5ynNEAki/view?usp=sharing>